

Embedding-Based Retrieval and Causal Chain Structural Mapping for Diagnosis and Prognosis of NTSB FAR 121 Aviation Accidents

Kanu Shetkar, Sankaran Mahadevan, Jesse Spencer-Smith
Vanderbilt University

Abstract

Aviation safety analysis has long relied on the structured coded fields in accident investigation reports. Bayesian network models built on this coded data have been effective at capturing causal relationships between events, but the unstructured narrative reports written by pilots, crews, and investigators are left out of the analysis even though they hold context that the coded fields cannot capture. In this paper, we extend the Bayesian network work of [Zhang and Mahadevan \(2021\)](#) on the National Transportation Safety Board (NTSB) FAR 121 accident database by using large language models (LLMs) to bring narrative information into the analysis. We first describe a pure LLM pipeline and the limitations we observed with it, including hallucination, inconsistent probabilities, and sensitivity to input phrasing. These limitations motivated a hybrid approach in which the LLM is used for semantic retrieval and clustering while the probability calculations are kept grounded in frequency counts taken from the actual data. The hybrid approach uses a conditional probability architecture based on the law of total probability across clustered incident types, and supports both diagnosis (backward inference from outcomes to causes) and prognosis (forward inference from precursors to downstream events). We then describe a structural mapping extension that adds a second similarity signal grounded in the underlying causal mechanism of each incident, and combines it with the embedding score through a single-line fusion formula to rerank the top retrieved candidates. We apply both methodologies to the same FAR 121 dataset from 1982 to 2016 used in the prior Bayesian network research, which lets us compare the two directly. On a held-out test set of 77 incidents, the structural mapping extension improves Top-1 diagnosis accuracy by 2.6 percentage points over the embedding-only baseline, with 2 strict wins and 0 strict losses on the binary outcome. The improvement is reproducible, statistically supported, and traceable to specific structural agreement between the query and the retrieved incidents.

1 Introduction

Over the past three decades, global commercial aviation has continued to grow at a fast pace. According to the International Air Transport Association ([IATA, 2019](#)), commercial airline carriers move close to one billion passengers within the United States each year. The consequences of an aviation accident are severe, and this makes the analysis of past accidents an important tool for preventing future ones. Reports collected over the past several decades give us a record we can study to understand how small abnormal events grow into accidents.

The NTSB accident database holds two kinds of information for every incident. The first is structured data: coded event sequences, occurrence codes, subject codes, findings, injury data, and aircraft damage data. These fields use a fixed vocabulary, which makes them well suited for statistical models. The second is unstructured data: free-form narratives written by pilots, crews, and investigators, along with the probable cause statements written by NTSB analysts. These

narratives carry context that the coded fields cannot capture, such as a pilot reporting an unusual vibration during climb, or a comment about weather conditions and human factors that did not get coded into a fixed category. The structured data gives consistency, and the narratives give context. The work in this paper aims to bring both into the same analysis.

Zhang and Mahadevan (2021) applied a Bayesian network to the NTSB aviation accident database, focusing on FAR 121 accidents from 1982 to 2016. Their network captures the causal relationships in the event sequences and supports both forward inference, which predicts outcomes from causes, and backward inference, which identifies likely causes from observed outcomes. Bayesian networks of this type require predefined nodes and probability tables, so they can only operate on the coded fields. The narratives are left out of the model, even though they often contain the detail that determines how a given accident actually unfolded. Before LLMs, bringing narrative text into the analysis would have required manual conversion of every sentence into a coded form, which is not practical at the scale of the full FAR 121 dataset.

Recent progress in large language models has changed this. LLMs can read narrative text directly, find similar past incidents based on meaning rather than keywords, and produce structured outputs that downstream code can use. This opens the door to bringing the narrative data into the analysis without throwing away the rigor of the existing Bayesian framework. In our initial experiments we used an LLM end-to-end for both retrieval and probability estimation. That approach turned out to have several problems. The LLM produced probabilities that did not match the underlying frequencies in the data, made confident claims that were not supported by any cited incident, and changed its answer when the wording of the input query changed by a small amount. We summarize these issues in Section 1.1 and they are the reason we moved to a hybrid design in which the LLM is used only for the parts it does well.

In this paper we describe two methodologies that build on Zhang and Mahadevan (2021). The first is a hybrid approach in which the LLM handles narrative retrieval and clustering, and frequency-based counts handle the probability calculations through the law of total probability. The second is a structural mapping extension that adds a second similarity signal based on the causal chain of each incident and uses it to rerank the top candidates returned by the hybrid pipeline. Both methodologies are applied to the same FAR 121 dataset from 1982 to 2016 used in the prior Bayesian network work, which lets us compare the two approaches directly.

1.1 Limitations of the Pure LLM Approach

Before settling on the hybrid design, we built a pipeline in which an LLM was responsible for the entire reasoning process: retrieving similar past cases, extracting contextual factors from the narratives, scoring and ranking candidate causes, and producing recommendations. Three problems with this approach made it unsuitable for the task. First, the LLM does not produce probabilities in the way a Bayesian network does. It infers relationships from learned language patterns rather than from explicit frequency counts, so the numbers it returns are not grounded in the actual data and are not reproducible across runs. Second, the LLM sometimes produced confident outputs that were not supported by any incident in the database, a behavior often called hallucination. Even with retrieval grounding, the model still tended to overstate certainty. Third, the system was sensitive to small changes in how the query was phrased, which is not acceptable for a safety analysis tool. From these issues we drew the lesson that LLMs are good at retrieving and structuring narrative text, and frequency-based counts taken from the actual data are good at producing reliable probabilities, and these two parts should be kept separate. The hybrid approach in Section 5 is built on this lesson.

1.2 Contributions

The contributions of this paper are summarized as follows.

1. We extend the Bayesian network work of [Zhang and Mahadevan \(2021\)](#) by bringing the narrative reports in the NTSB FAR 121 database into the analysis through a hybrid pipeline that combines LLM-based retrieval with frequency-based probability calculations using the law of total probability.
2. We apply the hybrid pipeline to both diagnosis (cause identification from a query incident) and prognosis (next-event prediction from a defining event), using the same FAR 121 dataset from 1982 to 2016 used in the prior work.
3. We add a structural mapping extension that extracts a structured causal chain for each incident, computes a structural similarity score using sequence alignment, and combines this score with the embedding cosine through a single-line fusion formula to rerank the top retrieved candidates.
4. We evaluate the structural mapping extension on a held-out test set of 77 FAR 121 incidents, using paired statistical tests, and show a 2.6 percentage point improvement in Top-1 diagnosis accuracy with 2 strict wins and 0 strict losses on the binary outcome.

The rest of the paper is organized as follows. Section 2 reviews related work in aviation safety analysis, structure-mapping theory, and two-stage retrieval. Section 3 describes the NTSB FAR 121 accident data and the fields used in this work. Section 4 introduces the background concepts that the methodologies rely on, including embeddings, k-means clustering, the law of total probability, and the Needleman-Wunsch sequence alignment algorithm. Section 5 presents the hybrid LLM methodology in detail, and Section 6 presents the structural mapping extension. Section 7 works through a full numerical example of the hybrid pipeline for both diagnosis and prognosis, and Section 8 reports the results of the structural mapping extension on the held-out test set, including statistical analysis and case studies. Section 9 compares the two approaches, and Section 10 concludes with contributions, limitations, and future work.

2 Related Work

This work draws on three lines of prior research: Bayesian and statistical models of aviation accident data, structure-mapping theory in cognitive science, and two-stage retrieval with reranking in information retrieval.

2.1 Aviation Accident Analysis

The most direct prior work for this paper is the Bayesian network model of [Zhang and Mahadevan \(2021\)](#), which builds a directed graphical model of the NTSB FAR 121 accident database and supports both forward (cause to outcome) and backward (outcome to cause) inference. That model relies entirely on the structured coded fields, since a Bayesian network needs a fixed node set and discrete conditional probability tables. [Zhang and Mahadevan \(2019\)](#) earlier applied ensemble machine learning models to a related aviation incident risk prediction task, again using structured features only. Our work extends this line by bringing the narrative reports into the analysis, while keeping the probability calculations in the spirit of the prior work by grounding them in frequency counts taken from the same FAR 121 data.

2.2 Structure-Mapping Theory

The structural mapping extension in this paper operationalizes ideas from structure-mapping theory in cognitive science. [Gentner \(1983\)](#) introduced structure-mapping as a theoretical framework for analogical reasoning, with the central claim that deep similarity between two situations is about the relational structure (which objects play which roles, in what causal order), not the surface attributes of the objects. [Gentner and Markman \(1997\)](#) sharpened this view by separating analogy (matching on relational structure with little surface overlap) from literal similarity (matching on both relational structure and surface attributes), and argued that human similarity judgments give more weight to systematic relations than to first-order relations or attributes. [Falkenhainer et al. \(1989\)](#) developed the Structure-Mapping Engine (SME), an algorithm that searches for the best mapping between two structured representations and assigns evidence values to candidate match hypotheses, with higher-order structure providing trickle-down support to lower-level matches. [Goldstone \(1994\)](#) showed that human similarity judgments incorporate alignment and feature comparison together rather than as independent stages. [Spencer-Smith and Goldstone \(1997\)](#) showed that the weights given to different features during similarity comparison shift dynamically with the comparison context, which is consistent with the role-system-mechanism weighting in our step similarity formula. We do not implement SME directly. We implement a small, domain-tuned version of the same idea: a fixed three-field schema (role, system, mechanism), a per-step similarity that weights role most heavily, and a chain score that classifies aligned pairs into strong, partial, and unmapped categories.

2.3 Two-Stage Retrieval and Reranking

The fusion step in Section 6.6 fits inside the broader pattern of two-stage retrieval in information retrieval. [Nogueira and Cho \(2019\)](#) introduced a two-stage architecture in which a fast first-pass retriever (BM25 or a similar index-based scorer) returns a candidate set of passages and a second, more expensive scorer reranks them using a learned model. The structural mapping extension uses the same shape: the embedding cosine plays the role of the fast first-pass score and the structural similarity plays the role of the slower per-pair reranker. The alignment step itself uses the dynamic programming algorithm of [Needleman and Wunsch \(1970\)](#), which was originally developed for global alignment of two protein sequences and is now standard for any two-sequence alignment task with a per-position similarity function and a gap penalty.

3 NTSB Accident Investigation Data

The National Transportation Safety Board (NTSB) is an independent federal agency, created in 1967, that investigates accidents and incidents across all modes of transportation in the United States, including aviation, highway, rail, marine, and pipeline. The work in this paper focuses on the aviation portion of the database, and within aviation we focus on FAR 121 air carriers, which cover large U.S.-based airlines, regional air carriers, and all-cargo operators. This is the same population used by [Zhang and Mahadevan \(2021\)](#), which lets us compare results directly with the prior Bayesian network work.

The FAR 121 portion of the NTSB database from 1982 to 2016 contains 2,243 accident and incident reports, each spread across 14 separate files. The files cover aircraft characteristics, engine characteristics, sequences of events, occurrences, findings, injury data, and the narrative reports written by pilots, crews, and investigators. For this work we merge the 14 files into a single record per incident, keyed by the NTSB event ID, so that the structured fields and the narratives for a

given incident can be retrieved together. The merged dataset preserves the same 2,243 incidents as the prior work and adds nothing new at the row level. The narrative text is what is new from a modeling perspective, since the prior Bayesian network work only used the coded fields.

Two structured fields are used heavily in the methodology that follows. The first is the sequence of events for each incident, which is an ordered chain of occurrences with one occurrence marked as the defining event. The defining event acts as the anchor for the prognosis task: the events before the anchor are the precursors, and the events after the anchor are the downstream consequences. The second is the findings field, which lists the causes and contributing factors identified by the NTSB investigators, and which we use as the ground truth for the diagnosis task. The narrative text supports the retrieval step in the methodology and is also used in the structural mapping extension to extract a structured causal chain for each incident.

4 Background Concepts

This section introduces the four background concepts that the methodologies in Section 5 and Section 6 build on. The goal is to fix notation and give the minimum needed to follow the rest of the paper.

4.1 Vector Embeddings and Cosine Similarity

A vector embedding maps a piece of text to a fixed-length vector of real numbers in a way that places texts with similar meaning close together in the vector space. We use OpenAI’s `text-embedding-3-small` model, which produces 1,536-dimensional vectors. Each dimension does not have a single human-interpretable meaning, but together the dimensions capture topic, phase of flight, type of failure, and other latent features of the text.

Once the texts are mapped to vectors, the similarity between any two texts can be computed mathematically. The standard measure is cosine similarity. Given a query embedding $\mathbf{q}$ and an incident embedding $\mathbf{i}$, the cosine similarity is

$$\text{sim}(\mathbf{q}, \mathbf{i}) = \frac{\mathbf{q} \cdot \mathbf{i}}{\|\mathbf{q}\| \|\mathbf{i}\|}, \quad (1)$$

where $\mathbf{q} \cdot \mathbf{i}$ is the dot product and $\|\cdot\|$ is the Euclidean norm. The dot product measures the directional overlap between the two vectors, and dividing by the magnitudes normalizes the result so that the score depends on direction rather than length. The output ranges from 0, meaning the two texts are unrelated, to 1, meaning the two texts have identical meaning. For example, the query *loss of engine power during cruise flight* returns a similarity of 0.81 against the incident *turbine blade separation during cruise* and only 0.25 against *bird strike on ground*, because the first is semantically related and the second is not.

4.2 K-means Clustering

K-means is an unsupervised clustering algorithm that partitions a set of data points into k groups, where each point is assigned to the group whose centroid it is closest to. The algorithm minimizes the sum of squared distances between each point and its assigned centroid. We use k-means with $k = 50$ on the 2,243 incident embeddings to organize the database into 50 incident-type clusters during a one-time setup step. Each cluster is then given a short text label produced by GPT-4o-mini, based on the incidents that fall in the cluster, so that downstream code can refer to clusters by name (for example, *turbine blade failure* or *engine fire*).

4.3 Law of Total Probability

The law of total probability gives a way to compute the marginal probability of an event from a set of conditional probabilities and their associated weights. If C is the event of interest, $K_1, K_2, \dots, K_m$ is a partition of the sample space, and Q is a query, then

$$P(C \mid Q) = \sum_{j=1}^m P(C \mid K_j) P(K_j \mid Q). \quad (2)$$

In Section 5, C will be a candidate cause, K_j will be one of the clusters from Section 4.2, and Q will be a query incident. The conditional terms $P(C \mid K_j)$ are estimated as cluster-level frequency counts, and the weights $P(K_j \mid Q)$ are computed from the cosine similarities between the query and the incidents in each cluster.

4.4 Needleman-Wunsch Sequence Alignment

Needleman-Wunsch is a dynamic programming algorithm originally developed for aligning two protein sequences (Needleman and Wunsch, 1970). The algorithm finds the best global alignment of two sequences by filling a two-dimensional table dp with the best score reachable at each (i, j) position. At each cell, the algorithm chooses the best of three moves: align position i in sequence A with position j in sequence B , skip a position in A , or skip a position in B . The recurrence is

$$dp[i][j] = \max \begin{cases} dp[i-1][j-1] + \text{sim}(A[i], B[j]) & \text{align,} \\ dp[i-1][j] + g & \text{skip in } A, \\ dp[i][j-1] + g & \text{skip in } B, \end{cases} \quad (3)$$

where $\text{sim}(A[i], B[j])$ is the per-position similarity score and g is a small negative gap penalty that allows but discourages skipping. The final cell $dp[n][m]$ holds the best score for aligning the full two sequences. We use this algorithm in Section 6 to align the structured causal chains of two incidents, where each position in the sequence is one step in the causal chain.

5 Phase 2 Methodology: Hybrid LLM Approach

This section describes the hybrid pipeline that combines LLM-based retrieval and clustering with frequency-based probability calculations using the law of total probability. The pipeline supports both diagnosis and prognosis, and both modes share the same retrieval and clustering steps in Sections 5.1 through 5.4. The two modes diverge at the probability step, which is described separately for diagnosis in Section 5.5 and for prognosis in Section 5.6. A full numerical walk-through for each mode is given later in Section 7.

5.1 Merged Dataset

The 14 NTSB files described in Section 3 are merged into a single record per incident, keyed on the NTSB event ID. Each merged record holds the structured fields (events, occurrences, subject codes, findings, injury and damage data) and the narrative text for that incident in one place. The merged dataset is stored as JSON for fast lookup. This step is one-time and offline. At query time the system only reads from the merged file, which avoids the cost of cross-referencing multiple files for every incident.

5.2 Embeddings

Each incident in the merged dataset is converted into a set of embedding vectors using OpenAI’s `text-embedding-3-small` model, which produces 1,536-dimensional vectors as defined in Section 4.1. We embed the narrative text, the findings, and the entries from the NTSB event dictionary (the `ct_seqevt.txt` file that maps numerical event codes such as 1452 to their English meanings). We embed all three of these because the system needs to be able to match a free-form query against either the free narrative text, the structured cause findings, or the standardized event descriptions. In total, embedding the 2,243 incidents produces 4,360 embedding vectors, since each incident contributes multiple text pieces. The result is a searchable embedding index over the full database. A user query is embedded with the same model so that it lives in the same vector space as the indexed incidents.

5.3 Similarity Search

Given an embedded query, the system retrieves the most relevant past incidents using the cosine similarity from Equation 1. The query embedding is compared against all 4,360 embeddings in the index, and the top 50 most similar incidents are kept for the next step. Fifty was chosen as a compromise between coverage (enough incidents to support cluster-level frequency counts) and focus (few enough that the retrieved set still concentrates on the query topic). For example, the query *loss of engine power during cruise flight* retrieves *turbine blade separation during cruise* at 0.81, *engine failure and fire in flight* at 0.80, *compressor stall during climb* at 0.71, and *bird strike on ground* at 0.25. Only the first three pass the relevance bar and would appear in the top 50; the bird strike example illustrates the kind of incident the cosine score correctly pushes out.

5.4 Clustering

The top 50 retrieved incidents are not a single homogeneous group. They are a mix of failure scenarios that may have very different causal patterns and downstream consequences. To organize them, we use the pre-computed k-means clusters described in Section 4.2. During the one-time setup step, k-means with $k = 50$ was run on all 2,243 incident embeddings, and each cluster was given a short label produced by GPT-4o-mini. These cluster labels are stored directly in the merged dataset, so at query time the system only needs to look up the cluster of each retrieved incident. For the example query *loss of engine power during cruise flight*, the top 50 incidents distribute as follows: 12 fall in the *turbine blade failure* cluster, 5 in the *engine fire* cluster, 5 in the *ice ingestion* cluster, and the remainder spread across smaller clusters. The cluster assignments are the input to the probability step in the next two sections.

5.5 Diagnosis

In diagnosis mode, the goal is to estimate the probability of each candidate cause given the query, $P(\text{Cause} \mid Q)$. The cluster organization from Section 5.4 lets us write this as a sum across clusters using the law of total probability, with each cluster contributing in proportion to how relevant it is to the query.

5.5.1 Cluster-Level Cause Probabilities

For each cluster K_j , we compute the cluster-level cause distribution $P(\text{Cause} \mid K_j)$ by counting how often each cause appears in the incidents that belong to that cluster. The causes for each incident

come from the NTSB findings field, where we use natural language processing to extract the causal phrases recorded by the investigators (for example, *fuel contamination*, or *failure of the hydraulic system*). Each incident may list more than one cause, since aviation accidents are often the result of multiple contributing factors. For one cluster, we go through each incident in the cluster, extract every cause mentioned, and count how often each unique cause appears. Dividing each count by the total number of cause mentions in the cluster gives the cluster-level cause probability. The same procedure is repeated for every cluster, so that every cluster ends up with its own probability distribution over causes.

5.5.2 Cluster Weights

The cluster weight $P(K_j | Q)$ measures how relevant cluster K_j is to the query. We compute the unnormalized cluster weight as the average cosine similarity of the incidents from K_j that appear in the top 50, multiplied by the number of such incidents:

$$w_j = \bar{s}_j \cdot n_j, \quad (4)$$

where $\bar{s}_j$ is the mean cosine similarity of the incidents from cluster K_j that appear in the top 50, and n_j is the count of such incidents. The two factors capture the two ways a cluster can be relevant: it can have many retrieved incidents, or it can have a few highly similar ones, or both. The weights are then normalized to a probability distribution

$$P(K_j | Q) = \frac{w_j}{\sum_k w_k}. \quad (5)$$

5.5.3 Combining Across Clusters

The cluster-level cause probabilities from Section 5.5.1 and the cluster weights from Section 5.5.2 are combined using the law of total probability from Equation 2:

$$P(\text{Cause} | Q) = \sum_j P(\text{Cause} | K_j) P(K_j | Q). \quad (6)$$

This produces a probability distribution over candidate causes, and the most probable cause is returned as the system’s primary diagnosis. As a small inline example, if the cluster weight $P(\text{turbine blade failure} | Q)$ is 0.56 and the cluster-level cause probability $P(\text{improper maintenance} | \text{turbine blade failure})$ is 0.35, then the contribution of this cluster to $P(\text{improper maintenance} | Q)$ is $0.56 \times 0.35 = 0.196$. A full multi-cluster walk-through with real numbers is given in Section 7.

5.6 Prognosis

In prognosis mode, the question is reversed. Given a current event, the system predicts what is most likely to happen next. If a pilot reports a loss of engine power, the system returns a probability distribution over the events that historically follow that condition, so that preventive actions can be considered before the next event happens. The prognosis pipeline reuses Sections 5.1 through 5.4 unchanged. The only difference is what is extracted from the retrieved incidents.

5.6.1 Sequence of Events Data

For each incident, the NTSB sequence of events field gives an ordered chain of occurrences in the order they happened. One occurrence in the chain is marked as the defining event, which is the

main event that triggered the NTSB investigation. The defining event acts as the anchor: the events before it are the precursors, and the events after it are the downstream consequences. For prognosis, we are interested in the downstream events. For each retrieved incident, we find the defining event in its sequence and record what comes immediately after.

5.6.2 Cluster-Level Next-Event Probabilities

For each cluster K_j we compute the cluster-level next-event distribution $P(\text{Next Event} \mid K_j)$ by counting how often each event appears as the immediate successor of the defining event across the incidents in that cluster. The procedure has five steps. First, we take one cluster, for example *turbine blade failure* with 12 incidents. Second, for each of those incidents, we find the defining event in its sequence. Third, we record what event comes immediately after the defining event. Fourth, we count the frequency of each next event across the cluster. Fifth, we divide each count by the total number of sequences contributed by the cluster to get the cluster-level next-event probability. The same procedure is repeated for every cluster.

5.6.3 Combining Across Clusters

The same form of the law of total probability used for diagnosis applies to prognosis, with the next event in place of the cause:

$$P(\text{Next Event} \mid Q) = \sum_j P(\text{Next Event} \mid K_j) P(K_j \mid Q). \quad (7)$$

The cluster weights $P(K_j \mid Q)$ are computed in the same way as in Section 5.5.2. The output is a probability distribution over candidate next events, and the most probable one is returned as the predicted next event.

5.6.4 Multi-Step Prediction

Aviation incidents do not stop at one event. Events cascade in a sequence: A leads to B , B leads to C , and so on. To predict events further down the chain, we use transition probabilities. A transition probability is a conditional probability over sequential events, $P(\text{Event}_t \mid \text{Event}_{t-1})$, that tells us what tends to follow each event. Once we know the distribution at step $t - 1$, the distribution at step t is given by another application of the law of total probability:

$$P(\text{Event}_t \mid Q) = \sum_e P(\text{Event}_t \mid \text{Event}_{t-1} = e) P(\text{Event}_{t-1} = e \mid Q). \quad (8)$$

The transition probabilities are estimated from all consecutive pairs in the sequence data across the retrieved incidents. For example, if *engine shutdown* is followed by *fire or smoke* in 7 out of 20 sequences in the data, then $P(\text{fire or smoke} \mid \text{engine shutdown}) = 7/20 = 0.35$. This step can be applied repeatedly to extend the prediction further down the chain, with one extra application of Equation 8 per additional step.

6 Phase 3 Methodology: Structural Mapping Extension

The hybrid approach in Section 5 relies on text similarity to decide which past incidents are relevant to a new query, and then on cluster-level frequency counts to combine evidence across the retrieved set. This works well when the wording of the new incident lines up with the wording of the cause

finding it should match. The approach has a known blind spot, which we describe in Section 6.1, and the structural mapping extension in this section is built specifically to address it. The extension is an operationalization of the structure-mapping framework of Gentner (1983) on the NTSB data, with the alignment carried out by the dynamic programming algorithm of Needleman and Wunsch (1970) and the reranking step following the two-stage retrieval pattern of Nogueira and Cho (2019).

6.1 Motivation

Two incidents with very different causal mechanisms can read similar in plain text, and two incidents with the same underlying mechanism can be described in very different vocabulary. The phrase *loss of engine power during cruise* can match against an incident caused by fuel contamination, an incident caused by a maintenance error on a turbine blade, or an incident caused by ice ingestion. The embedding does not know which of these mechanisms the new query actually belongs to. It only knows that the words sound similar. Gentner and Markman (1997) make the same point in general terms: matching on words and matching on causal structure are two different things, and the structural match is the one that tells you whether two cases are really alike. The structural mapping extension addresses this by adding a second similarity signal that compares the underlying causal mechanism of two incidents, not just their wording, and uses it to rerank the top retrieved candidates.

6.2 Causal Chain Schema

For every incident we extract a structured causal chain. The chain is a list of steps in the order they occurred, and each step has three fields. The *role* field captures where the step sits in the causal sequence, with allowed values *initiating event*, *propagation*, *system compromise*, *system failure*, *terminal failure*, *operational consequence*, and *outcome*. The *system* field captures which subsystem the step belongs to, with allowed values such as *engine mechanical*, *lubrication*, *fuel*, *hydraulic*, *electrical*, *structural*, *flight controls*, *landing gear*, *propulsion*, *fire protection*, *environment*, *human performance*, *maintenance*, and *aircraft*. The *mechanism* field captures what physically went wrong at that step, with allowed values such as *maintenance error*, *material degradation*, *fatigue*, *corrosion*, *contamination*, *thermal damage*, *mechanical loosening*, *deprivation*, *overheating*, *fire*, *separation*, *power loss*, *loss of control*, *terrain collision*, *procedural error*, and *design deficiency*.

In addition to the chain itself, each incident has a list of contributing factors and a single failure pattern label such as *thermal cascade*, *maintenance latent defect cascade*, or *human performance chain*. The schema is fixed so that two incidents extracted independently can be compared field by field, which is what allows the alignment step in Section 6.5 to produce a meaningful score.

6.3 Extraction with the LLM

The structured chain is extracted by sending the incident text to GPT-4o-mini with a fixed schema prompt. The model returns a JSON object that we then normalize against the allowed values listed in Section 6.2. Anything outside the allowed list is coerced to a default value such as *unknown*, so that the downstream scoring code is never surprised by an unexpected category. This is the only place in the pipeline where an LLM is involved beyond the retrieval embedding step. The extraction is run once per incident in the training set and once per query, and the result is written to a cache file. There is no LLM call during ranking, scoring, or fusion.

6.4 Step Similarity

To compare two causal chains we first need a way to compare two individual steps. Given step A from one chain and step B from another, we compute a step similarity score

$$\text{sim}(A, B) = 0.50 \cdot r(A, B) + 0.30 \cdot m(A, B) + 0.20 \cdot s(A, B), \quad (9)$$

where r , m , and s are the role, mechanism, and system component scores. The weights (0.50 for role, 0.30 for mechanism, 0.20 for system) reflect the relative importance of each field and follow the relational hierarchy of [Gentner \(1983\)](#), in which systematic relations get more weight than first-order relations or attributes. Role gets the highest weight because it captures where in the causal sequence the step sits, which is the most diagnostic field for distinguishing one mechanism from another. Mechanism gets the next-highest weight because it describes the causal connection between consecutive steps. System gets the lowest weight because two incidents can involve different physical components and still share the same underlying failure pattern. [Spencer-Smith and Goldstone \(1997\)](#) showed that human similarity judgments shift the weights given to different features depending on the comparison context, which provides further support for treating these three fields as distinct contributions rather than as a single concatenated bag of words. If two steps have the exact same role, r returns 1.0. If the roles are different but adjacent in the causal sequence (for example, *propagation* and *system compromise*), r returns a value between 0.7 and 0.95 from a small lookup table. The system and mechanism scores work the same way, with a compatibility table that gives partial credit for related categories such as *fuel* and *engine mechanical*, or *fatigue* and *material degradation*. The output is a single number between 0 and 1 that measures how close the two steps are.

6.5 Chain Alignment

Two incidents do not always have the same number of steps in their chain, and the steps do not always appear in the same order. To compare the two full chains we use the Needleman-Wunsch alignment algorithm from Section 4.4, with the per-step similarity from Equation 9 as the alignment score and a constant gap penalty of $g = -0.05$ so that skipping a step is allowed but discouraged. After the dynamic programming table is filled, the final cell holds the best score for aligning the two full chains.

We then walk the alignment back and classify each aligned position into one of three categories. If the step similarity is at least 0.70, the pair counts as a strong match. If it is between 0.35 and 0.70, it counts as a partial match. Aligned pairs below 0.35 and any positions left as gaps in the alignment count as unmapped. The structural similarity is then computed as

$$\text{struct_sim} = \frac{\text{strong} + 0.5 \cdot \text{partial}}{|A| + |B|} - 0.10 \cdot \frac{\text{unmapped}}{|A| + |B|}, \quad (10)$$

where $|A|$ and $|B|$ are the lengths of the two chains being compared, so the denominator is the total number of step positions across both chains. The strong/partial split is a small, domain-tuned version of the evidence framework used in the Structure-Mapping Engine ([Falkenhainer et al., 1989](#)), where match hypotheses receive partial evidence values and structural agreement at one level provides trickle-down support to lower-level matches. The unmapped penalty term subtracts a small fraction proportional to the number of positions that did not find a good match, so that two chains with many gaps cannot reach a high score on the strength of a few matched steps alone. The final structural similarity is a number between 0 and 1.

6.6 Score Fusion

The structural similarity does not replace the embedding score. It reweights it. For each candidate incident, the new score is

$$\text{new_score} = \max(0, \text{cos_sim}) \cdot \exp(\alpha \cdot \text{struct_sim}), \quad (11)$$

where cos_sim is the cosine similarity from Section 5.2, struct_sim is the structural similarity from Section 6.5, and α is a tuning parameter that controls how strongly the structural signal influences the ranking. The two-stage shape of fast first-pass retrieval followed by a slower per-pair reranker is the standard pattern of Nogueira and Cho (2019); the embedding cosine plays the role of the fast first pass and the structural similarity plays the role of the slower reranker. Combining the two scores into a single ranking score, rather than running them as fully independent stages, is consistent with Goldstone (1994), which showed that human similarity judgments incorporate alignment and feature comparison together rather than as independent stages. Based on initial experiments, we set $\alpha = 2.0$. When the structural similarity is 0, the exponential factor is 1 and the new score reduces to the embedding cosine. The fusion can never make a candidate look more similar than it already was on text alone unless the structural signal also agrees. The new scores are sorted, and the top candidate becomes the prediction returned by the system. As a small inline example, a candidate with cosine 0.62 and structural similarity 0.78 has a new score of $0.62 \cdot \exp(2 \cdot 0.78) = 2.96$, while a candidate with the same cosine 0.62 and structural similarity 0.40 has a new score of $0.62 \cdot \exp(2 \cdot 0.40) = 1.38$. The first candidate gets a much larger boost, so it moves above the second in the ranking.

7 Verification: Phase 2 Worked Example

This section walks through a full numerical example of the hybrid pipeline. Section 7.1 works through the diagnosis mode end to end on a sample query, and Section 7.2 works through the prognosis mode on the same query.

7.1 Diagnosis Walkthrough

The query for this walkthrough is *loss of engine power during cruise flight*. The top 50 most similar incidents have been retrieved using cosine similarity, as in Section 5.3, and the cluster assignments have been looked up from the pre-computed clusters, as in Section 5.4. For clarity, the worked example below uses only the top three clusters. The same procedure runs for every cluster in the full retrieval, but the smaller set is enough to illustrate the calculation.

The top three clusters and their summary statistics are: *turbine blade failure* with 12 incidents and average cosine similarity 0.82; *engine fire* with 5 incidents and average cosine similarity 0.79; and *ice ingestion* with 5 incidents and average cosine similarity 0.75. We first compute the cluster weights using Equation 4.

$$\begin{aligned} w_{\text{turbine}} &= 0.82 \times 12 = 9.84, \\ w_{\text{fire}} &= 0.79 \times 5 = 3.95, \\ w_{\text{ice}} &= 0.75 \times 5 = 3.75. \end{aligned}$$

The total weight is $9.84 + 3.95 + 3.75 = 17.54$. Normalizing each cluster weight by the total gives

the cluster probabilities:

$$\begin{aligned} P(\text{turbine blade failure} \mid Q) &= 9.84/17.54 = 0.56, \\ P(\text{engine fire} \mid Q) &= 3.95/17.54 = 0.23, \\ P(\text{ice ingestion} \mid Q) &= 3.75/17.54 = 0.21. \end{aligned}$$

The turbine blade failure cluster gets the largest weight because it has both the most incidents and the highest average similarity.

For each cluster, we then compute the cluster-level cause probabilities $P(\text{Cause} \mid K_j)$ using the procedure in Section 5.5.1. As one example, consider the engine failure cluster, which contains 12 incidents in this query’s top 50. We extract every cause mentioned in those 12 incidents and end up with 20 total cause mentions, since some incidents list more than one cause. Across the 20 mentions, *improper maintenance* appears 7 times, *fuel contamination* appears 5 times, *manufacturing defects* appears 3 times, and *pilot error* appears 2 times. Dividing each count by 20 gives

$$\begin{aligned} P(\text{improper maintenance} \mid \text{engine failure}) &= 7/20 = 0.35, \\ P(\text{fuel contamination} \mid \text{engine failure}) &= 5/20 = 0.25, \\ P(\text{manufacturing defects} \mid \text{engine failure}) &= 3/20 = 0.15, \\ P(\text{pilot error} \mid \text{engine failure}) &= 2/20 = 0.10. \end{aligned}$$

The same procedure runs for every cluster, so each cluster ends up with its own probability distribution over candidate causes.

We now combine the cluster weights with the cluster-level cause probabilities using the law of total probability from Equation 6. The output is a probability distribution over candidate causes, and the cause with the highest probability is the system’s primary diagnosis. The same procedure runs over the full top 50 retrieval, not only the three clusters shown here, but the structure of the calculation is the same.

7.2 Prognosis Walkthrough

Using the same query and the same top three clusters, we now work through the prognosis mode end to end. The target event for this walkthrough is *engine shutdown*: the question is, given the current condition described by the query, what is the probability that the next event is engine shutdown?

We first compute the cluster-level next-event probabilities $P(\text{engine shutdown} \mid K_j)$ using the procedure in Section 5.6.2. As an illustration, take the *turbine blade failure* cluster, which contains 12 incidents in the top 50. Of those 12, 10 have a clear defining event with at least one event after it. We go through each of those 10 incidents, find the defining event, and record what comes immediately after. Across the 10 sequences, *engine shutdown* appears 4 times, *powerplant malfunction* appears 3 times, *fire or smoke non-impact* appears 2 times, and *loss of engine power* appears 1 time. Dividing each count by 10 gives

$$\begin{aligned} P(\text{engine shutdown} \mid \text{turbine blade failure}) &= 4/10 = 0.40, \\ P(\text{powerplant malfunction} \mid \text{turbine blade failure}) &= 3/10 = 0.30, \\ P(\text{fire or smoke} \mid \text{turbine blade failure}) &= 2/10 = 0.20, \\ P(\text{loss of engine power} \mid \text{turbine blade failure}) &= 1/10 = 0.10. \end{aligned}$$

The same procedure runs for the engine fire and ice ingestion clusters. Suppose the within-cluster probabilities of engine shutdown for the three clusters are 0.40, 0.50, and 0.00 respectively.

We now apply the law of total probability from Equation 7, using the cluster weights computed in Section 7.1.

$$\begin{aligned} P(\text{engine shutdown} \mid Q) &= 0.40 \times 0.56 + 0.50 \times 0.23 + 0.00 \times 0.21 \\ &= 0.224 + 0.115 + 0.000 \\ &= 0.339. \end{aligned}$$

The probability that engine shutdown is the next event is 33.9%. The turbine blade failure cluster contributes the most, both because it has the largest cluster weight ($P(K \mid Q) = 0.56$) and because engine shutdown is a common downstream event in that cluster ($P(\text{shutdown} \mid K) = 0.40$). The engine fire cluster also contributes meaningfully even though it has fewer incidents, because engine shutdown is an even more common downstream event in that cluster (0.50). The ice ingestion cluster contributes nothing in this case because engine shutdown does not appear as a next event in that cluster’s sequences. The same calculation runs for every candidate next event, which produces a complete probability distribution over what could happen next after the query event.

To extend the prediction one more step forward, we apply Equation 8. Continuing the example above, suppose the transition probability from *engine shutdown* to *fire or smoke* is $7/20 = 0.35$ based on the consecutive pairs in the retrieved sequences. The contribution of the engine-shutdown branch to the probability of *fire or smoke* two steps out is $0.35 \times 0.339 = 0.119$. Repeating this across every step-1 event and summing the contributions gives the full step-2 distribution. The same procedure can be applied again for step 3, with one extra application of Equation 8 per additional step.

8 Verification: Phase 3 Results and Case Studies

This section reports the results of the structural mapping extension on a held-out test set, including aggregate metrics, statistical tests, distribution analysis, and case studies that explain the mechanism behind the wins and the losses. Section 8.1 describes the evaluation setup, Section 8.2 reports aggregate metrics, Section 8.3 reports the statistical analysis, Section 8.4 analyzes the distributions and rank behavior, Section 8.5 works through the cases where the structural mapping extension was correct and the embedding baseline was wrong, and Section 8.6 works through the cases where the structural mapping extension produced a lower match score than the baseline.

8.1 Evaluation Setup

We split the FAR 121 incident set into 177 training incidents and 77 held-out test incidents. The training set is used to build the structural cache, the embedding index, and the cluster-level frequency tables that feed the law of total probability. The test set is never seen during cache construction, retrieval index building, or any step of the pipeline. All metrics reported in this section are computed on the 77 held-out incidents.

We compare two conditions on the same 77 test incidents. Condition A0 is the embedding-only baseline from Section 5, with no structural reranking. Condition A2 is the structural mapping extension from Section 6, with the fusion step applied to the top retrieved candidates. The paired design lets us use the McNemar test and bootstrap intervals on per-incident differences, which are more powerful than treating the two conditions as independent samples.

The diagnosis metric is the same M1 score used throughout this work. For each test incident, the system’s top predicted cause is compared to the official cause findings written by NTSB investigators. The prediction passes if the embedding cosine similarity between the prediction and any

of the true cause findings is at least 0.75. Top-1 accuracy is the fraction of the 77 test incidents that pass at rank 1. Recall at 5 is the fraction where any of the top five predictions pass. The average match percent is the mean of the cosine similarity between the top-1 prediction and the closest true cause finding, expressed as a percentage.

The evaluation in this paper is restricted to the diagnosis task. Prognosis under the structural reranker is left to future work in Section 10.3.

8.2 Aggregate Results

Table 1 reports the aggregate diagnosis results for the embedding-only baseline (A0) and the structural mapping extension (A2) on the 77 test incidents. Figure 1 shows the same numbers as a bar chart with 95% Wilson confidence intervals on the rate metrics.

Table 1: Aggregate diagnosis performance on the 77 held-out FAR 121 test incidents. A0 is the embedding-only baseline. A2 is the structural mapping extension. Δ is the difference in percentage points.

Metric	A0	A2	Δ
Top-1 accuracy	32.5%	35.1%	+2.6 pp
Recall @ 5	54.5%	57.1%	+2.6 pp
Avg match %	63.31%	64.63%	+1.32 pp
Top-1 error rate	67.5%	64.9%	−2.6 pp

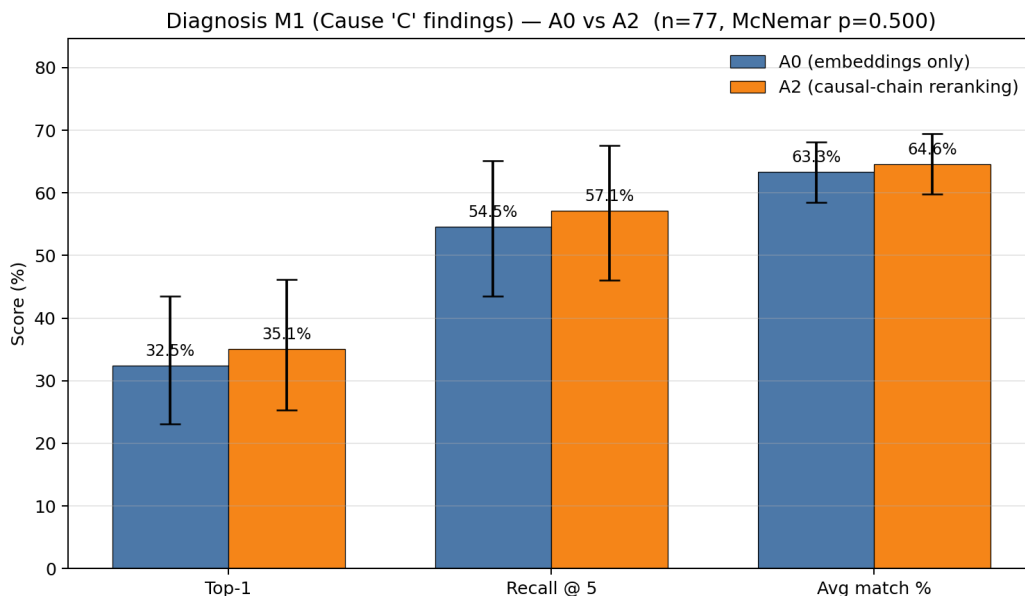


Figure 1: Top-1 accuracy, Recall at 5, and average match percent for A0 and A2 on the 77 held-out test incidents, with 95% Wilson confidence intervals on the rate metrics. McNemar exact two-sided p -value shown in the title.

The structural mapping extension improves on every metric. The improvement on Top-1 accuracy is +2.6 percentage points, which corresponds to going from 25 correct test incidents under A0

to 27 correct test incidents under A2.

8.3 Statistical Analysis

Because the test set is small ($n = 77$) and the differences are modest, we ran three statistical checks on the results. The full intervals are reported in Table 2.

First, we computed Wilson confidence intervals on each individual rate. Wilson is the standard interval for binomial proportions and behaves well at moderate sample sizes. Second, we ran McNemar’s exact test on the paired binary outcome (the M1 hit indicator) for the 77 incidents. McNemar is the right test for this comparison because A0 and A2 are evaluated on the same incidents, so the per-incident outcomes are paired. Third, we computed bootstrap 95% confidence intervals on the per-metric difference between A0 and A2, using 10,000 paired resamples.

Table 2: Statistical intervals for the four diagnosis metrics. Wilson intervals are reported on the rate metrics. Difference intervals are bootstrap 95% confidence intervals on the paired per-incident difference, with 10,000 resamples.

Metric	A0	A0 95% CI	A2	A2 95% CI	Difference 95% CI
Top-1 accuracy	32.5%	[23.1, 43.5]	35.1%	[25.3, 46.2]	[0.0, +6.5] pp
Recall @ 5	54.5%	[43.5, 65.2]	57.1%	[46.0, 67.6]	[0.0, +6.5] pp
Avg match %	63.31%	—	64.63%	—	[+0.18, +2.87] pp

The bootstrap interval on the difference excludes zero for the average match percent and lies entirely on the non-negative side of zero for Top-1 accuracy and Recall at 5. Under repeated sampling the direction of the improvement is consistent across all three metrics: the lower bound on every paired difference is at or above zero.

The McNemar paired flip table is the strongest single piece of evidence in this section, and is shown in Figure 2. Of the 77 test incidents, 25 were correct under both methods, 50 were wrong under both, 2 were correct only under A2, and 0 were correct only under A0. The McNemar exact two-sided p -value is 0.500, which is above the conventional 0.05 cutoff, but the split itself is the best possible outcome for a paired comparison: A2 never made things worse. Every incident where the two methods disagreed went in A2’s favor. There is not a single test incident where the structural reranking knocked a previously correct answer below the threshold. The p -value sits at 0.500 because there are only 2 disagreements in total, not because the effect is weak. The Wilson intervals on the rates, the bootstrap intervals on the continuous metrics, and the one-sided flip pattern in the McNemar table all point the same way.

8.4 Distribution and Rank Behavior

To check whether the improvement is concentrated in a few extreme cases or spread across the test set, we plotted the distribution of the top-1 match score for both methods. Figure 3 shows the result. The A2 distribution has more mass above the 0.75 threshold and the mean shifts from 63.3% to 64.6%. So A2 is not just flipping a couple of edge cases. The whole distribution shifts up, with extra weight landing in the high-similarity bins above the threshold.

We also plotted the rank position of the first passing prediction under each method. Figure 4 shows the result. Under A2, the first passing prediction lands at rank 1 for 27 incidents (versus 25 under A0), and two fewer incidents have no passing prediction in the top 5 (33 versus 35). This is the rank shift behind the Recall at 5 and average match percent improvements: the structural

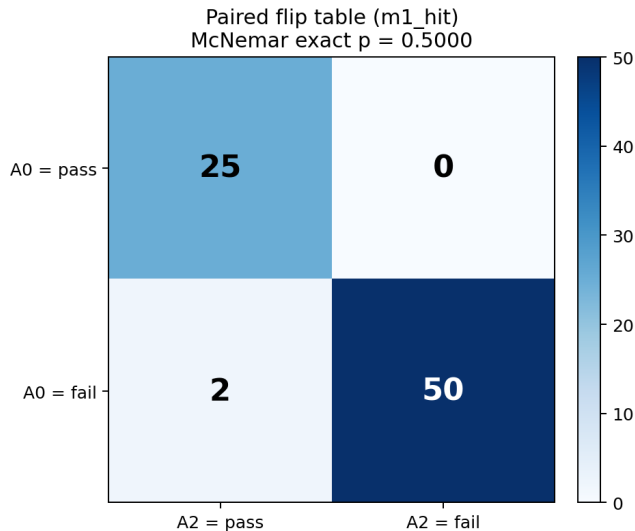


Figure 2: McNemar paired flip table on the binary M1 outcome at the 0.75 cosine threshold for the 77 test incidents. Both correct: 25. Both wrong: 50. A2 correct only: 2. A0 correct only: 0. McNemar exact two-sided $p = 0.500$.

reranking is pulling correct answers higher up the ranked list, which is the intended behavior of a reranker.

8.5 Case Studies: Strict Wins

The aggregate statistics show that A2 is better on average, but they do not by themselves explain why the reranking moves things in the way it does. This section walks through the two cases where A2 was correct and A0 was wrong, and shows what changed at the cause-text level. Table 3 summarizes the two strict wins.

Table 3: The two test incidents where A2 was correct and A0 was wrong. Match % is the top-1 cosine similarity score between the predicted cause and the closest true cause finding.

Incident ID	A0 match %	A2 match %	Δ
20100114X11754	42.2	76.8	+34.6 pp
20120802X03552	58.5	87.1	+28.6 pp

Both wins are large. The smaller gain is +28.6 percentage points and the larger is +34.6. In one of the two wins, A0 returned a generic environmental cause finding about clear-air turbulence affecting personnel, a phrase that appears many times in the corpus and lives in a part of the embedding space that pulls many narratives toward it. The pure embedding ranking falls into this attractor whenever the new query uses environmental words such as *cruise* or *descent*. In the other win, A0 returned a generic organizational cause about *equipment manufacture* that captures none of the underlying mechanism. The structural mapping extension is built to catch both failure modes: it looks past the wording and asks whether the candidate’s causal chain actually matches the query’s chain, so it can pull mechanism-matched candidates above the generic ones that only sound right.

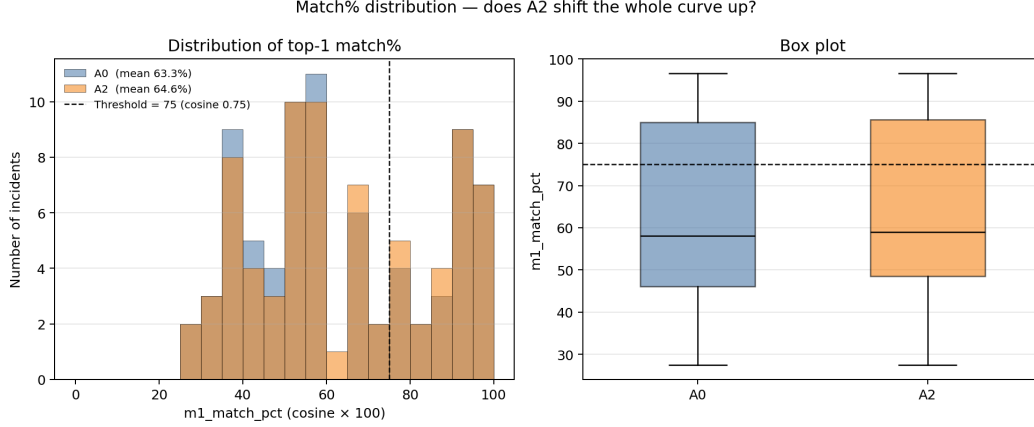


Figure 3: Histogram and box plot of the top-1 match score across the 77 test incidents, for A0 and A2. The dashed line at 75% marks the cosine threshold used to define a hit.

8.5.1 Worked Example: Incident 20100114X11754

The truth for this incident is *aircraft power plant, engine fuel and control, fuel distribution, failure*. A0 picked *organizational issues, development, manufacture/production, equipment manufacture, manufacturer* and scored 42.2% on the match metric, which is below the 0.75 threshold and counts as a miss. A2 picked *aircraft power plant, engine (turbine/turboprop), turbine section, failure* and scored 76.8%, which is above the threshold and counts as a hit.

The A0 pick is a generic organizational cause that says nothing about the physical mechanism that brought the engine down. The A2 pick is a specific power-plant turbine section failure, which sits in the same engine-failure mechanism family as the truth’s fuel-distribution failure. The structural reranker found this candidate because its extracted causal chain (fuel system material degradation, propagation to thermal damage, terminal engine failure) aligned with the query’s chain at the role and mechanism level, while the manufacturer candidate’s chain had almost no overlap with the query’s chain. The boost factor in Equation 11 for the turbine-section candidate was therefore much larger than for the manufacturer candidate, which moved it past A0’s pick in the ranking.

8.5.2 Worked Example: Incident 20120802X03552

The truth for this incident is *environmental issues, physical environment, object/animal/substance, animal/bird, effect on operation*. A0 picked *environmental issues, conditions/weather/phenomena, turbulence, clear air turbulence, effect on personnel* and scored 58.5%. A2 picked *environmental issues, physical environment, object/animal/substance, animal/bird, effect on equipment* and scored 87.1%, which differs from the recorded truth by a single word (*equipment* vs *operation*) and is correct in mechanism.

The two candidates are both environmental causes, but only one matches the underlying mechanism of the query. The bird-strike candidate’s causal chain (initiating event in the environment, propagation to aircraft systems, terminal effect on equipment) lines up with the query’s chain at every step. The clear-air turbulence candidate’s chain matches the query only on the initiating-event role, and diverges on mechanism. The structural similarity for the bird candidate was therefore larger than for the turbulence candidate, the boost factor was correspondingly larger, and the bird candidate moved to rank 1.

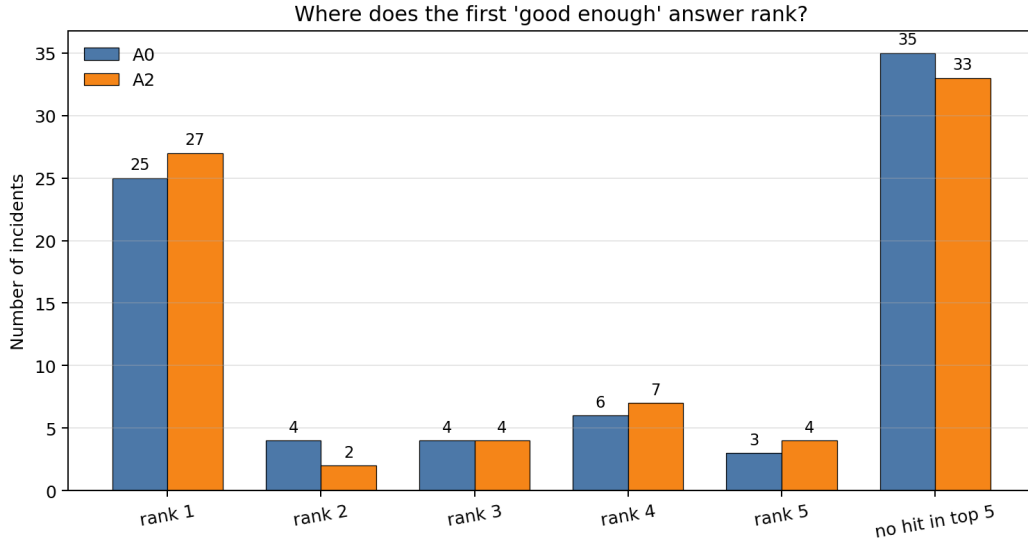


Figure 4: Rank position of the first passing prediction under each method across the 77 test incidents. Bars show the count of incidents whose first passing prediction lands at rank 1, 2, 3, 4, 5, or has no hit in the top 5.

8.5.3 Demotion Mechanics

A useful subtlety about the fusion formula in Equation 11 is that it never literally lowers any candidate’s score. The factor $\exp(\alpha \cdot \text{struct_sim})$ is at least 1 for non-negative structural similarity, so every candidate’s score either stays the same or goes up under A2. What A2 does is raise everyone’s score, but raise some candidates more than others. A candidate with low structural similarity gets a small boost. A candidate with high structural similarity gets a large boost. The low-boost candidate gets passed in the ranking by the high-boost candidate. That is what we mean when we say A2 demoted a candidate. In the bird-strike case in Section 8.5.2, the turbulence candidate’s score went up under A2, but the bird-strike candidate’s score went up more, so the turbulence candidate fell from rank 1 to a lower rank. The worst case for a candidate is when its extracted chain has nothing in common with the query. Its structural similarity is 0, so its boost factor is $\exp(0) = 1$ and its score does not change at all under A2. Meanwhile every other candidate with real chain agreement gets multiplied up and passes it in the ranking.

8.6 Case Studies: Losses

For completeness we also report the cases where A2’s continuous match score is lower than A0’s. Fourteen test incidents fall into this group. Table 4 shows the four largest drops; the remaining ten are all at or below one tenth of a percentage point and are essentially numerical noise.

Two observations support the no-regression claim from Section 8.3. First, the two largest drops are on incidents that were already failing under A0, so the structural reranking did not break a case the baseline was getting right; it just left a wrong answer wrong with a slightly different score. Second, the next two drops are tiny (-0.1 percentage points) and occur on incidents that pass the 0.75 threshold under both methods, so the reranking did not push them across the threshold either. There is not a single test incident where A0 was correct at the 0.75 threshold and A2’s reranking knocked the answer below the threshold. This is the same finding that the McNemar

Table 4: Test incidents where A2’s match percent is lower than A0’s, sorted by the size of the drop. None of these drops crosses the 0.75 hit threshold.

Incident ID	A0 match %	A2 match %	Δ	A0 hit	A2 hit
20131220X70905	43.9	41.2	−2.8 pp	no	no
20170308X73155	37.2	36.2	−1.0 pp	no	no
20090921X73211	81.6	81.5	−0.1 pp	yes	yes
20110417X11718	76.1	76.0	−0.1 pp	yes	yes

table in Section 8.3 shows in a different form.

8.6.1 Worked Example: Incident 20131220X70905

The truth for the largest A2 drop is *aircraft systems, equipment/furnishings, cargo compartments, incorrect use/operation*. A0 picked *environmental issues, conditions/weather/phenomena, turbulence, convective turbulence, effect on personnel* and scored 43.9%. A2 picked *organizational issues, development, manufacture/production, equipment manufacture, manufacturer* and scored 41.2%. Both predictions are wrong on the binary outcome and the gap between them is under three percentage points.

The query is a cargo-compartment incorrect-operation incident, which involves a human-action mechanism on an aircraft systems subsystem. Neither candidate is a good match. The convective-turbulence candidate from A0 is in the wrong subsystem entirely (environment instead of aircraft systems) and is purely a passive environmental cause. The manufacturer candidate from A2 is in the right vicinity (an organizational cause that touches equipment) but does not capture the cargo-compartment operation mechanism either. The reranker pulled the manufacturer candidate ahead because its chain has slightly more in common with the query’s chain at the role level (both involve an organizational or human contribution), but the cosine similarity to the standardized truth phrasing dropped because the truth is written in aircraft-systems vocabulary. The drop is small in absolute terms (−2.8 percentage points) and on an incident that was never going to pass the threshold under either method, so it has no effect on the binary outcome.

The remaining drops in Table 4 are similar in character. The two −0.1 percentage point drops occur on incidents that pass the 0.75 threshold under both methods, so the structural reranking did not change the answer or the binary outcome on those cases. None of the 14 drops crosses the threshold in either direction.

9 Comparison and Discussion

The two methodologies in this paper sit at different points on a tradeoff between simplicity and discrimination power. The hybrid approach in Section 5 is the simpler of the two: it relies only on text embeddings, k-means clusters, and frequency counts, and it produces probability distributions that can be read off directly. It works well as long as the wording of the query lines up with the wording of the cause finding it should match. The structural mapping extension in Section 6 adds a one-time per-incident extraction step and a single-line reranking formula on top of the hybrid pipeline. It does not change any of the upstream retrieval or clustering steps, and it does not replace the embedding score; it only reweights the top candidates based on causal mechanism agreement.

The verification results in Section 8 show that the structural mapping extension improves Top-1 diagnosis accuracy by 2.6 percentage points on the held-out test set, with 2 strict wins and 0

strict losses on the binary outcome. The aggregate improvement is moderate, but the shape of the improvement is the more important finding. Every disagreement between the two methods went in the structural mapping extension’s favor at the binary level. The cases where the extension won were large wins, +28.6 and +34.6 percentage points on the continuous match score. The cases where the extension lost were small, with the largest drop at -2.8 percentage points and most under one tenth of a percentage point, and never broke a case the baseline was getting right. This is the signature of a real signal rather than noise. The case studies in Sections 8.5 and 8.6 trace each disagreement to the candidate cause text and the underlying chain agreement, so the changes can be explained one incident at a time and not only as an aggregate statistic.

The hybrid approach is the right starting point when the retrieval set is large and the cause vocabulary is varied, because it is fast, transparent, and grounded in counts. The structural mapping extension is the right add-on when the retrieval set contains incidents whose wording is similar but whose underlying mechanism is not. In that case, the embedding cosine alone is not enough to separate the relevant from the irrelevant, and the structural signal does the work of breaking the tie.

10 Conclusion

10.1 Contributions

This paper makes four contributions to the analysis of the NTSB FAR 121 accident database. First, we extend the Bayesian network work of [Zhang and Mahadevan \(2021\)](#) by bringing the narrative reports into the analysis through a hybrid pipeline that combines LLM-based retrieval and clustering with frequency-based probability calculations using the law of total probability. Second, we apply the hybrid pipeline to both diagnosis and prognosis, using the same FAR 121 dataset from 1982 to 2016 used in the prior work, and we provide a full numerical walk-through of both modes. Third, we add a structural mapping extension that extracts a structured causal chain for each incident, computes a structural similarity score using Needleman-Wunsch sequence alignment, and combines this score with the embedding cosine through a single-line fusion formula to rerank the top retrieved candidates. Fourth, we evaluate the structural mapping extension on a held-out test set of 77 FAR 121 incidents using paired statistical tests, and we trace each win and each loss to the actual structural agreement values that drove the decision, so the result can be explained at the level of individual incidents and not only as an aggregate statistic.

10.2 Limitations

Several limitations should be addressed in future work. First, the test set size ($n = 77$) is modest, which limits the power of the McNemar test and widens the confidence intervals on the rate metrics. A larger held-out set, or k-fold cross validation across the FAR 121 set, would give tighter intervals. Second, the structural extraction step depends on an LLM, and errors in extraction (such as a wrong system or mechanism label) propagate into the alignment and produce noisy structural similarities. We mitigate this by normalizing all extracted values against a fixed allowed list, but the schema is finite and may not cover every nuance of every incident. A schema with finer-grained subsystem labels would likely improve the structural signal. Third, the fusion parameter α in Equation 11 was set to 2.0 based on initial experiments. A grid search on a held-out subset would allow α to be picked from data instead of by hand, which may push the gain higher. Fourth, the M1 match score measures cosine similarity between the predicted cause text and the NTSB cause text, which is itself a proxy for what we care about. In cases where the structural mapping

extension picks a candidate with strong mechanism agreement but different wording, the metric can punish the prediction even when the answer may be more useful for an investigator. A metric that scores mechanism agreement directly would likely show A2 doing better than the match score alone suggests.

A second observation about the metric concerns its sensitivity at the threshold. The Top-1 hit metric is binary at 0.75 cosine similarity, so a candidate at 0.74 counts as a miss while a candidate at 0.76 counts as a hit. Some of the rank movement that the structural reranker produces is invisible to this metric: a candidate that goes from rank 5 to rank 1 with a match score that stays below 0.75 does not flip the binary outcome even though it is a meaningful improvement to a user reading the ranked list. The average match percent is more sensitive in this respect, and shows a consistent positive shift under A2.

10.3 Future Work

There are three places this work can go next. First, the structural reranker can be carried over to the prognosis pipeline in Section 5.6. The reranker plugs into the same retrieval stage that prognosis already uses, with a small change to the cluster-weight formula in Equation 4 so that the weights are computed from the fused score rather than the raw cosine. The next-event prediction task can then be evaluated under A2 the same way diagnosis was evaluated in Section 8. Second, the per-step similarity weights in Equation 9 were set by hand at 0.50, 0.30, and 0.20, following the relational hierarchy of Gentner (1983). These weights are grounded in theory rather than tuned on data, and can be learned from the held-out set in future work, which would let the fusion adapt to the structure of the data instead of relying on a fixed prior. Third, a final stage that scores the candidate’s predicted cause text against the query’s narrative directly, in addition to the current structural and cosine signals, would address the failure mode in Section 8.6.1 where mechanism agreement and text agreement diverge.

References

- B. Falkenhainer, K. D. Forbus, and D. Gentner. The structure-mapping engine: Algorithm and examples. *Artificial Intelligence*, 41(1):1–63, 1989.
- D. Gentner. Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, 7(2):155–170, 1983.
- D. Gentner and A. B. Markman. Structure mapping in analogy and similarity. *American Psychologist*, 52(1):45–56, 1997.
- R. L. Goldstone. Similarity, interactive activation, and mapping. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 20(1):3–28, 1994.
- International Air Transport Association. IATA forecast predicts 8.2 billion air travelers in 2037, 2019. <https://www.iata.org/pressroom/pr/Pages/2018-10-24-02.aspx>.
- S. B. Needleman and C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.
- R. Nogueira and K. Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- J. Spencer-Smith and R. L. Goldstone. The dynamics of similarity. *Cognitive Studies: Bulletin of the Japanese Cognitive Science Society*, 4:38–56, 1997.

- X. Zhang and S. Mahadevan. Ensemble machine learning models for aviation incident risk prediction. *Decision Support Systems*, 116:48–63, 2019.
- X. Zhang and S. Mahadevan. Bayesian network modeling of accident investigation reports for aviation safety assessment. *Reliability Engineering and System Safety*, 209:107371, 2021.